ENTERPRISE SOFTWARE ARCHITECTURE

Status: Active Staging & Production Ready

Eventura Platform

Eventura Enterprise: Complete Multi-Tenant Event, Competition & Innovation Management Operating System

Executive Specification Document: This document provides the comprehensive technical and operational blueprint for the Eventura Platform. Built specifically for universities, collegiate consortiums, technology communities, and hackathon organizers, Eventura unifies the full event lifecycle—from proposal drafting and administrative budget sign-offs to team formation, live QR gate attendance, fair rubric evaluation, automated AI executive dossiers, and tamper-proof digital credentials.

PROJECT NAME

Eventura Platform

SYSTEM VERSION

v1.2.0 (Production Handover)

DOCUMENT VERSION

v2.0 - Comprehensive Startup Specification

PREPARED BY

Lead Systems Architect & Core Engineering Team

LAST UPDATED

October 30, 2026

TARGET ARCHITECTURE

PostgreSQL 16 + Express + React 19 + Gemini AI

Document Structure & Master Index

01. Cover Page
02. Project Summary
03. Business Context
04. System Overview & Architecture Diagram
05. Technology Stack & Rationale
06. Major Modules & Responsibilities
07. End-to-End System Workflow
08. Data Model & Entity Relationships
09. External Integrations
10. Configuration & Environment Variables
11. Deployment Overview & Topology Diagram
12. Project Structure & Folder Hierarchy
13. Known Limitations & Technical Debt
14. Maintenance, Logging & Troubleshooting
15. Version Changelog
16. Companion References & Handover Manuals

2. Project Summary

2.1 Purpose of the Project

Eventura Platform (branded as Eventura) is an enterprise-grade, multi-tenant Event, Competition, Hackathon, and Innovation Management Platform engineered for higher educational institutions, collegiate consortiums, technology communities, and corporate hackathons. The primary purpose of Eventura is to eliminate the severe fragmentation of conventional event organization—which relies on disjointed Google Forms, WhatsApp groups, manual judging spreadsheets, paper budget approval requests, and non-verifiable paper certificates—by unifying all operational stages into a single, cohesive, role-governed digital operating system.

2.2 Business Problem It Solves

- **Approval Bottlenecks**: Paper proposal forms delayed for weeks across department offices.
- **Roster Chaos**: Uncontrolled member swaps and duplicate registrations on demo day.
- **Biased & Delayed Judging**: Manual paper scoring leading to tabulation delays and errors.
- **Gate Congestion**: Slow physical check-in lines causing bottleneck queues at venues.
- **Credential Forgery**: Unverifiable paper certificates vulnerable to tampering.
- **Post-Event Reporting Friction**: Weeks spent gathering stats for accreditation boards.

2.3 Intended Users

- **Institutional Admins & Principals**: Formal event sanctions, venue allocation, governance.
- **Event Managers & Officers**: Rubric design, track config, budget approvals.
- **Student / Faculty Coordinators**: Proposal drafting, team tracking, AI dossier creation.
- **Evaluators & Judges**: Real-time digital scorecards, qualitative reviews.
- **Volunteers**: High-speed QR gate pass scanning with camera.
- **Hackers & Participants**: Team formation, project delivery, verifiable credentials.

Current Operational Status: Production Handover & Active Staging Development
The core backend (Express + PostgreSQL + Prisma ORM) and frontend (React 19 + TanStack Router) are fully implemented, fortified with multi-tenant tenant isolation and RBAC. All functional workflows—including Stripe payment settlement, Google Gemini AI dossier synthesis, and offline-resilient QR gate check-ins—have passed automated smoke and multi-tenant stress tests.

3. Business Context

3.1 Why Was This Project Built?

Institutional hackathons and innovation symposiums have transitioned from informal club activities into massive multi-stakeholder summits involving thousands of competitors, corporate sponsors, intellectual property submissions, and substantial institutional budgets. Despite this growth, management methodologies remained obsolete. Universities frequently incurred audit penalties, delayed student awards, lost track of sponsor funds, and struggled to compile compliance dossiers for national accreditation bodies (such as NAAC, NBA, and ABET). Eventura was conceived to modernize collegiate event governance with the same rigor found in enterprise ERP systems.

3.2 Real-World Value Proposition

85%

Reporting Time Saved

Automated Gemini AI synthesis generates full institutional event dossiers in seconds rather than weeks.

100%

Certificate Authenticity

Every award and attendance certificate features a cryptographic SHA-256 serial code and public QR verify URL.

0.5s

Gate Check-In Speed

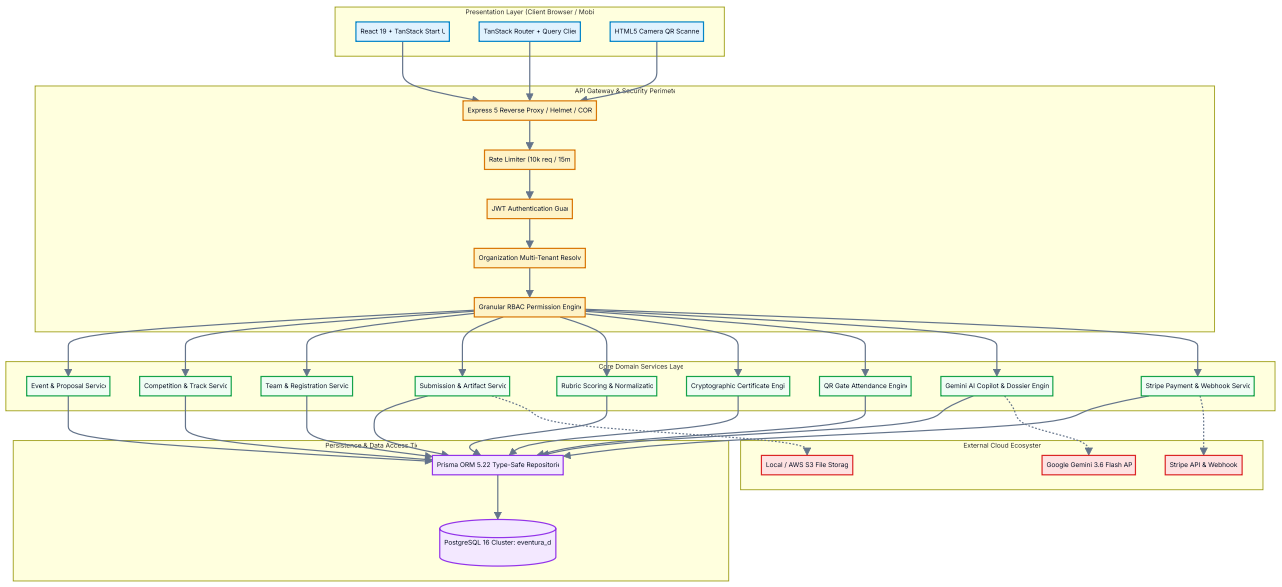
Sub-second mobile camera QR scanning eliminates bottleneck queues during registration check-ins.

4. System Overview

4.1 Application Description in Plain Terms

Eventura operates as a full-stack, multi-tenant Software-as-a-Service (SaaS) web application. Each university, college department, or company operates within its own strictly isolated organization realm (`organizationId`). When a user navigates to the platform, the client application executes role-based route matching. Behind the scenes, incoming requests pass through a multi-tier security perimeter that validates JWT authenticity, resolves active tenant membership, checks granular permissions (e.g., `events.create` , `evaluations.manage`), and applies tenant scoping at the repository level before interacting with PostgreSQL.

FIGURE 4.1: HIGH-LEVEL SYSTEM ARCHITECTURE & MULTI-TENANT PIPELINE ARCHITECTURAL MODEL



Unified full-stack architecture showing presentation, security perimeter, tenant-isolated services, database persistence, and external cloud integrations.

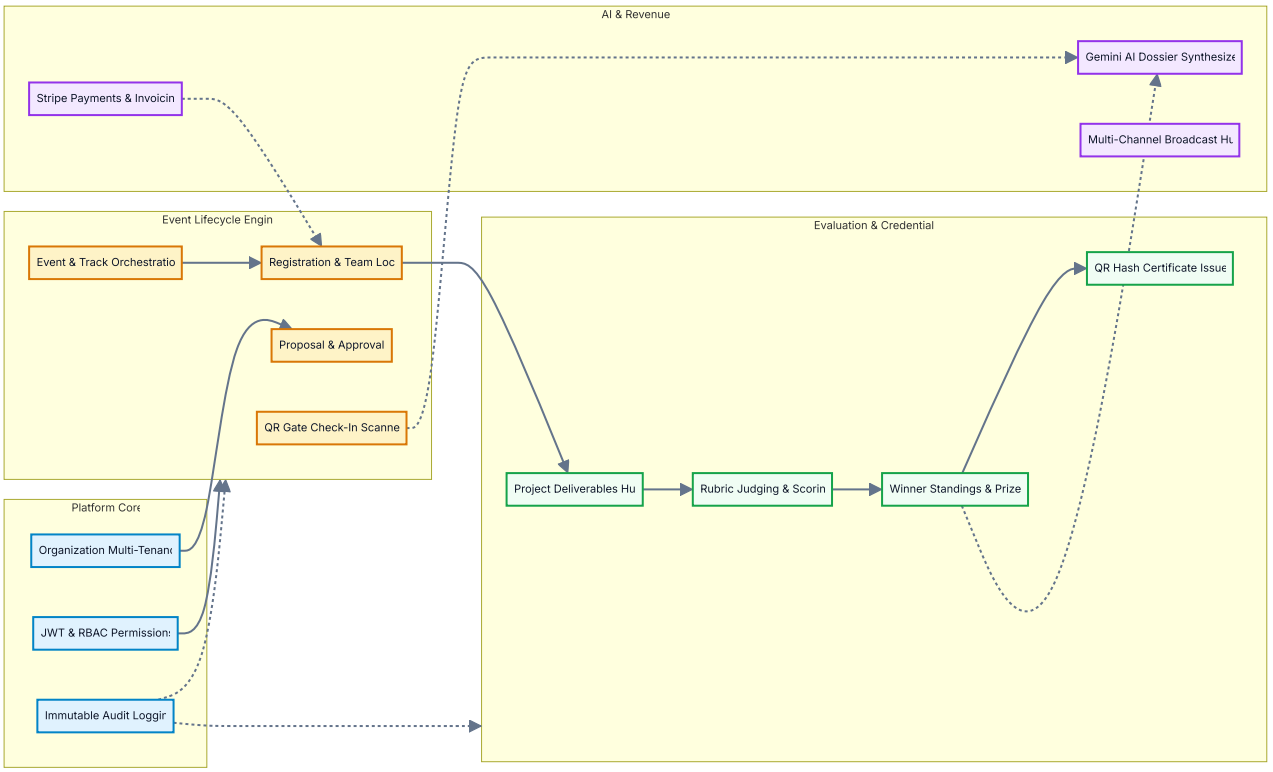
5. Technology Stack

The Eventura platform is built upon a modern, high-performance technology stack selected to maximize developer velocity, type safety, security compliance, and runtime predictability.

Layer	Technologies & Libraries	Technical Justification & Rationale
Frontend Core	React 19 · TypeScript 5.8 · Vite 8.2 · TanStack Start	React 19 provides concurrent rendering and optimized transitions. TypeScript enforces compile-time contract safety between client and server APIs. Vite guarantees sub-second Hot Module Replacement (HMR).
Client Routing & State	TanStack Router v1 · TanStack Query v5	100% type-safe search parameters and route definitions prevent broken links. TanStack Query manages automatic cache invalidation, background refetching, and zero-waterfall server state hydration.
Styling & UI Components	Tailwind CSS v4 · Radix UI · Lucide React · Recharts	Radix UI primitives deliver WAI-ARIA accessible component logic (dialogs, tooltips, accordions). Tailwind CSS v4 delivers lightweight atomic CSS compilation. Recharts provides responsive SVG dashboards.
Backend API	Node.js 22 LTS · Express 5.2 · TypeScript · tsx	Express 5 delivers native async promise rejection handling without unhandled crashes. Clean 4-tier separation: Route → Middleware → Controller → Service → Repository.
Validation & Security	Zod 3.24 · bcrypt 6.0 · jsonwebtoken 9.0 · Helmet 8.3	Zod enforces strict runtime schema validation across incoming HTTP payloads. bcrypt delivers salted password hashing. Helmet sets strict HTTP response security headers. Rate limiting defends against brute-force attacks.
Database & ORM	PostgreSQL 16 · Prisma ORM 5.22	PostgreSQL guarantees strict ACID transactions for payments and certificate serial generation. Prisma ORM generates auto-typed database models, declarative schema migrations, and relational joins.
AI Synthesizer	@google/generative-ai · Gemini 3.6 Flash	Google Gemini 3.6 Flash offers massive token context windows and ultra-fast generation speeds to transform raw event metrics into executive post-event dossiers and scoring summaries.
Payments & Hardware	Stripe API (v22.5) · html5-qrcode · PDFKit	Stripe handles multi-tier registrations and webhook-driven reconciliation. HTML5 QR scanner accesses phone camera directly. PDFKit compiles dynamic vector certificates and printable dossiers.

6. Major Modules

FIGURE 6.1: SUBSYSTEM DEPENDENCY & INTER-MODULE COUPLING **COMPONENT MAP**



1. Tenant & Organization Governance

Purpose: Provides tenant isolation and administrative boundaries for institutions.

Responsibilities: Tenant provisioning, subscription plan tracking, institutional domain binding, and tenant-scoped repository filtering.

Main Features: Multi-tenant database querying, custom branding settings, organization user directory.

Dependencies: PostgreSQL, Prisma Client, Auth Middleware.

2. Identity, Auth & RBAC Engine

Purpose: Secures all system endpoints and ensures least-privilege access.

Responsibilities: Salted password hashing, JWT signing/verification, permission matrix resolution, and session management.

Main Features: Granular permissions (`events.read`, `evaluations.manage`), token refresh, multi-role membership.

Dependencies: bcrypt, jsonwebtoken, Prisma User/Role tables.

3. Event Proposal & Approval Chain

Purpose: Enforces formal institutional governance before public launch.

Responsibilities: Multi-tier approval routing: Student Coordinator → Event Manager → Principal/Executive Director.

Main Features: Budget itemization, venue conflict detection, formal digital sign-off, transition to `APPROVED` state.

Dependencies: Event Service, Audit Logger, Notification Service.

4. Track & Competition Setup

Purpose: Manages specific hackathon problem statements and challenge domains.

Responsibilities: Rule definition, team size constraints (e.g. 2-5 members), prize allocation, and rubric linking.

Main Features: Multi-track events (AI, Web3, IoT), custom eligibility criteria, judging committee assignment.

Dependencies: Event Module, Rubric Evaluation Service.

5. Team Registration & Roster Lock

Purpose: Facilitates participant sign-up and prevents competition fraud.

Responsibilities: Invitation code generation, captain leadership transfer, member approval, and automated roster freeze.

Main Features: Automated locking at submission deadline, duplicate prevention, payment status binding.

Dependencies: Registration Repository, Stripe Payments.

6. Submission & Artifact Hub

Purpose: Collects and preserves competition deliverables.

Responsibilities: GitHub repository linking, demo pitch video URL, slide deck PDF storage, and immutable timestamping.

Main Features: Lockout upon timer expiration, file upload validation, version history tracking.

Dependencies: File Storage Service, Event Timer Service.

7. Rubric Evaluation & Scoring

Purpose: Eliminates subjective bias through structured digital scorecards.

Responsibilities: Weighted criteria evaluation, score normalization across judge panels, draft saving, and podium locking.

Main Features: Real-time leaderboard updates, blind evaluation modes, qualitative feedback logs.

Dependencies: Submission Repository, Judge Assignment Model.

8. QR Gate Attendance Scanner

Purpose: Accelerates physical check-ins during high-volume event mornings.

Responsibilities: Dynamic QR passport generation, camera scanner execution, offline scan caching, and double-entry prevention.

Main Features: Sub-second visual confirmation (Green/Red), session check-in logs, real-time turnout counter.

Dependencies: html5-qrcode, Attendance Service, User Passport.

9. Tamper-Proof Digital Certificates

Purpose: Issues permanent, verifiable credentials to attendees and winners.

Responsibilities: Dynamic vector PDF synthesis, SHA-256 cryptographic serial hashing, and public verification routing.

Main Features: Public verify screen (`/certificates/verify/:code`), one-click download, batch dispatch.

Dependencies: PDFKit, QRCode Generator, Certificate Repository.

10. Gemini AI Executive Dossier

Purpose: Synthesizes complete post-event accreditation dossiers automatically.

Responsibilities: Pulling event metrics, generating executive summaries, analyzing breakout projects, and noting challenges.

Main Features: Publication-ready PDF export with institutional stamps, faculty sign-off fields, zero-wait generation.

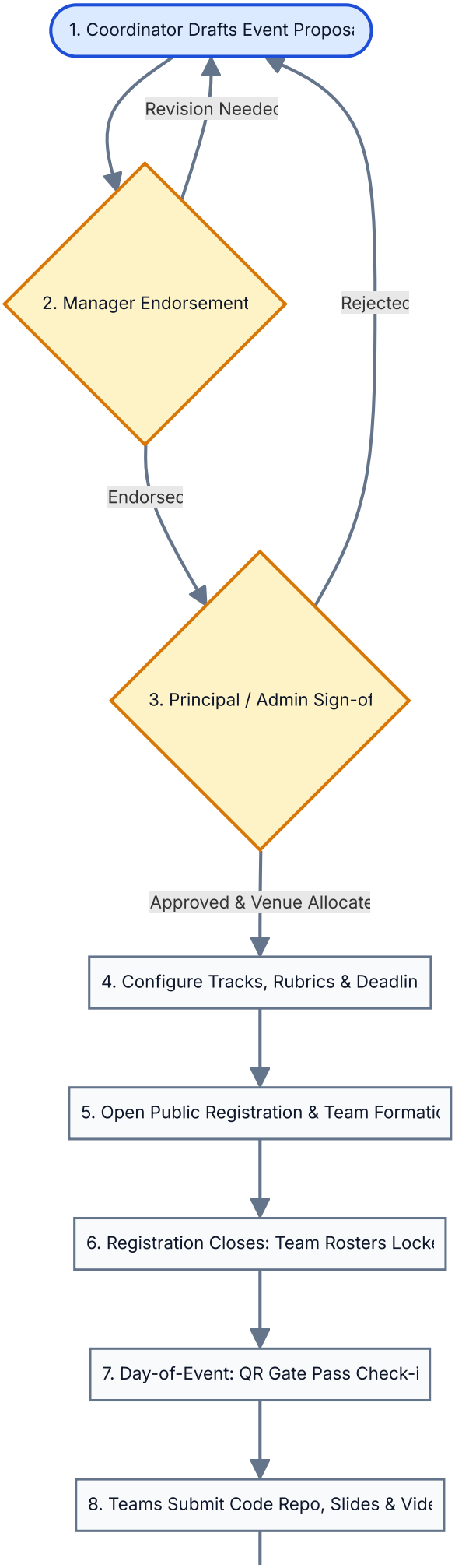
Dependencies: @google/generative-ai, Analytics Aggregator.

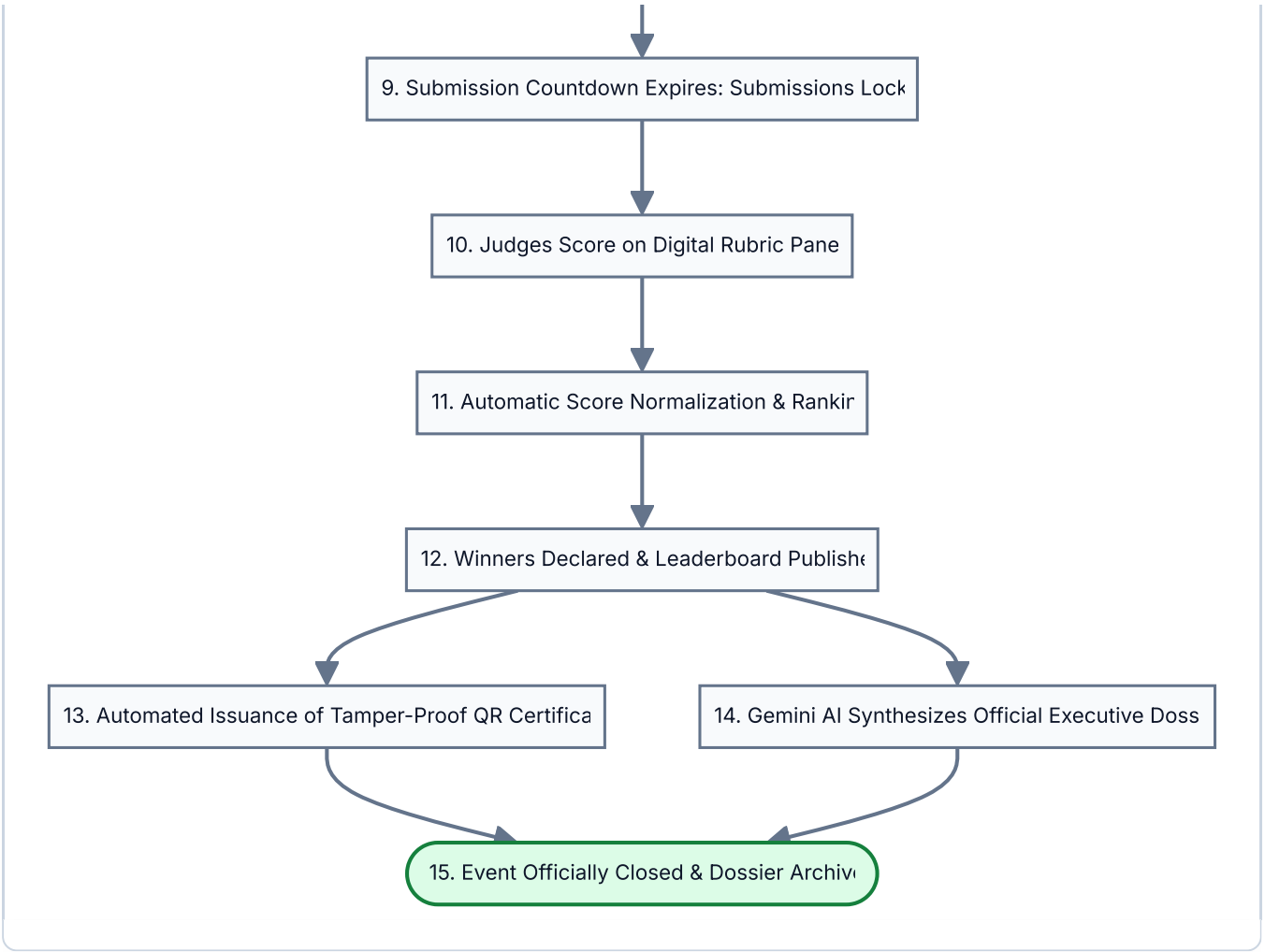
7. System Workflow

7.1 End-to-End Institutional Event Lifecycle

The Eventura platform enforces a 7-stage operational lifecycle that guarantees complete accountability, audit tracking, and operational fluidity from initial proposal to final dossier archival.

FIGURE 7.1: END-TO-END OPERATIONAL EVENT & EVALUATION WORKFLOW LIFECYCLE PROCESS FLOW


1. Coordinator Drafts Event Proposa
Revision Needed
2. Manager Endorsement
Rejected
Endorsed
3. Principal / Admin Sign-of
Approved & Venue Allocate
4. Configure Tracks, Rubrics & Deadlin
5. Open Public Registration & Team Formatic
6. Registration Closes: Team Rosters Locke
7. Day-of-Event: QR Gate Pass Check-i
8. Teams Submit Code Repo, Slides & Vide



7.2 Detailed Step-by-Step Workflow Description

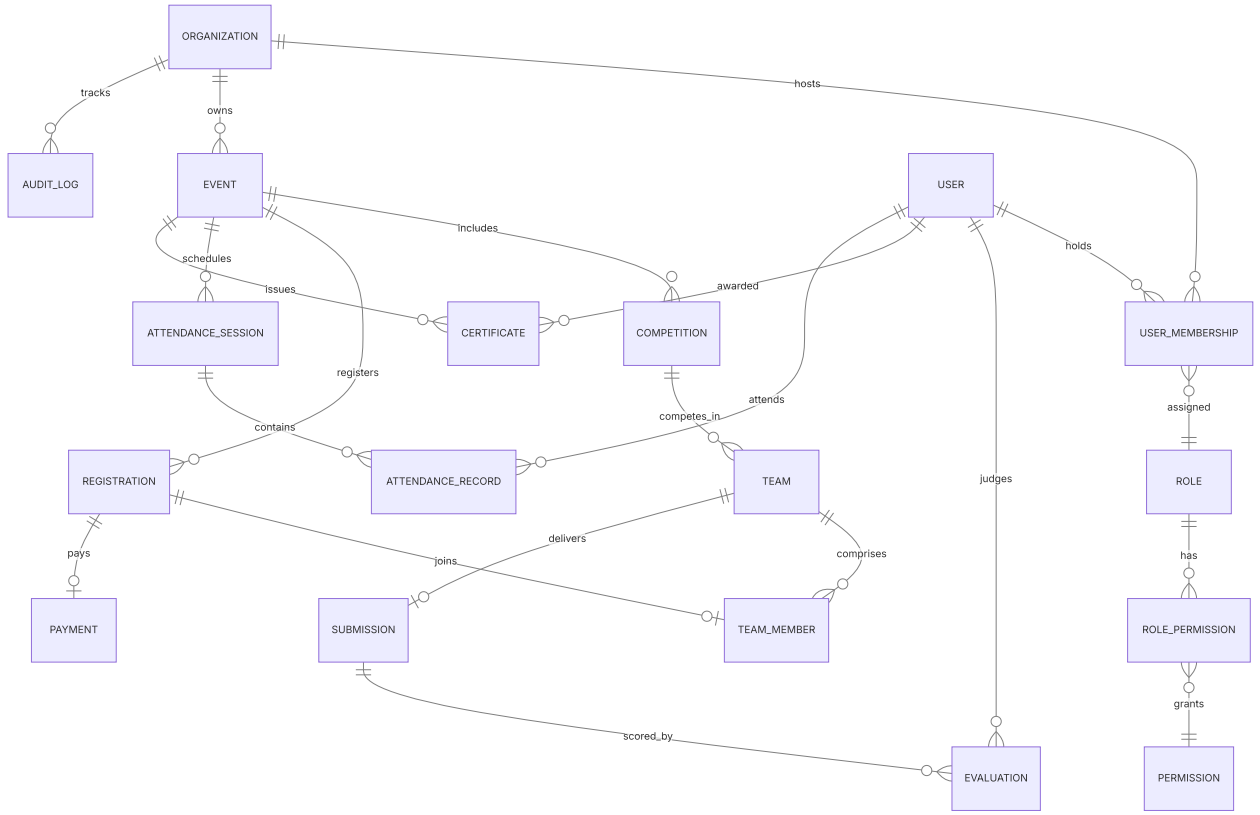
1. **Proposal Drafting (Student Coordinator)**: Coordinator drafts dates, target headcounts, itemized budget items, and required campus venues (e.g., Main Auditorium). Proposal is submitted with status `DRAFT` → `PENDING_REVIEW`.
2. **Manager Evaluation (Event Manager)**: Manager verifies budget feasibility, checks campus calendar for date collisions, and appends recommendations.
3. **Executive Sanction (Principal / Director)**: Highest authority clicks formal approval. System sets event status to `APPROVED`, locks venue reservation, and unlocks operational tracks.
4. **Track Configuration & Setup**: Manager sets problem statements, team bounds (2-5 hackers), judging rubric criteria (Innovation 40%, Technical Depth 30%, Viability 30%), and registration windows.
5. **Team Registration & Ticketing**: Hackers register, pay fees via Stripe (if ticketed), form teams with invite links. When deadline hits, team rosters freeze permanently.
6. **Gate Check-In**: Volunteers scan attendees' encrypted QR passes via mobile phone cameras. Gate logs update in real time; proxy attendance is prevented.
7. **Hackathon Submissions & Freeze**: Teams upload GitHub URLs, pitch decks, and live demo links. When the countdown hits 00:00:00, all submissions lock.
8. **Judging & Rubric Normalization**: Judges evaluate assignments on mobile scorecards. Scores are automatically normalized across panels to prevent outlier bias.
9. **Leaderboard & Winner Declaration**: Winners and podium standings are finalized. Cash prize allocations are recorded.
10. **Automated AI Dossier & Certificate Dispatch**: Gemini AI synthesizes the comprehensive post-event dossier with institutional formatting. Dynamic vector PDF certificates with SHA-256 QR codes are dispatched to all participants.

8. Data Model

8.1 Core Entities and Relationships

The Eventura database schema is implemented in PostgreSQL and modeled via Prisma ORM. Rather than exposing raw low-level database columns, the model below presents the core architectural entities and their cardinalities.

FIGURE 8.1: CORE ENTITY-RELATIONSHIP DIAGRAM — **DOMAIN DATA MODEL**



8.2 Entity Dictionary & Cardinality Highlights

Entity Name	Architectural Role & Key Attributes	Relational Cardinalities
`Organization`	Root tenant entity. Stores institutional name, slug, domain, subscription plan, and branding configurations.	1:N with UserMembership, Event, AuditLog.
`User`	Universal identity record. Contains credentials (hashed password), email, full name, avatar, and system state.	1:N with UserMembership, Evaluation, Certificate, AttendanceRecord.
`Role & Permission`	Granular RBAC model. Roles (e.g. Coordinator, Judge) map to permission grants (e.g. `events.update`).	M:N between Roles and Permissions via RolePermission.
`Event`	Central event container. Tracks lifecycle status (`DRAFT`, `APPROVED`, `COMPLETED`), budget, dates, and venue.	1:N with Competition, Registration, AttendanceSession, Certificate.
`Competition`	Specific competition track under an event (e.g., AI Track). Defines team size bounds, prizes, and criteria rubrics.	N:1 with Event; 1:N with Team.
`Team & Member`	Participant grouping. Manages team invite tokens, leadership, submission locks, and member rosters.	1:N with TeamMember; 1:1 with Submission; N:1 with Competition.
`Submission`	Deliverable package. Contains code repo URL, pitch deck PDF, demo video link, and submission lock timestamps.	1:1 with Team; 1:N with Evaluation.
`Evaluation`	Judge scoring instance. Contains criterion-specific scores, qualitative reviewer feedback, and normalized scores.	N:1 with Submission; N:1 with User (Judge).
`Certificate`	Cryptographic award/attendance document. Holds unique serial, verification hash, public URL, and PDF link.	N:1 with Event; N:1 with User.
`AttendanceSession`	Gate check-in period (e.g., Day 1 Morning Check-in). Tracks attendee arrival timestamps and volunteer IDs.	1:N with AttendanceRecord; N:1 with Event.
`Payment`	Financial transaction record. Stores Stripe session IDs, currency, amount, tax, and refund lifecycle states.	1:1 with Registration; N:1 with Organization.

9. External Integrations

Eventura seamlessly integrates with leading enterprise cloud services to deliver payments, artificial intelligence, QR scanning, and PDF compilation.

External Service	Integration Method	Core Purpose & Operational Value
Google Gemini AI	REST API via official SDK (`@google/generative-ai`)	Synthesizes executive post-event dossiers, summarizes competition project submissions, and auto-generates rubric evaluation feedback for participants.
Stripe Payments	REST API & Cryptographic Webhooks (`stripe` SDK)	Processes ticket checkout sessions, manages split disbursements for multi-department organizations, and automatically updates registration states via webhooks.
HTML5 QR Engine	Client-side Browser Camera (`html5-qrcode`)	Provides sub-second barcode and QR scanning directly through smartphone web browsers without requiring native mobile app installations.
PDFKit Generator	Server-side Binary Stream (`pdfkit`)	Compiles high-resolution, vector-accurate participant certificates with dynamic serial numbers, institutional signatures, and embedded verification QR codes.
Storage Providers	Local Storage & Pluggable S3 (`uploads/`)	Handles team presentation slide decks, event banners, and generated PDF dossiers. Implements an abstraction layer supporting AWS S3 or Cloudflare R2.

10. Configuration

10.1 Environment Variables (Sanitized)

Variable Name	Example Setting	Description & Requirement
`PORT`	`3000`	Port on which the Express backend HTTP server binds.
`NODE_ENV`	`development` / `production`	Node environment flag enabling optimizations and strict CORS.
`API_URL`	`http://localhost:3000/api/v1`	Public-facing base URL for backend REST API endpoints.
`VITE_API_URL`	`http://localhost:3000/api/v1`	Client-side build variable directing frontend fetch calls to API.
`FRONTEND_URL`	`http://localhost:8080`	Allowed CORS origin for browser client application.
`DATABASE_URL`	`postgresql://usr:pwd@host:5432/db`	Connection string for primary PostgreSQL database cluster.
`JWT_SECRET`	`[REDACTED_SECURE_SECRET]`	Cryptographic secret key used to sign and verify user JWTs.
`JWT_EXPIRES_IN`	`15m` (Access) / `7d` (Refresh)	Token lifetime configuration enforcing regular re-authentication.
`STORAGE_PROVIDER`	`local` / `s3`	Storage driver flag selecting filesystem or object cloud bucket.
`STRIPE_SECRET_KEY`	`sk_live_...` / `sk_test_...`	Stripe API secret key for payment intent and session creation.
`STRIPE_WEBHOOK_SECRET`	`whsec_...`	HMAC secret used to verify Stripe webhook event signatures.
`GEMINI_MODEL`	`gemini-3.6-flash`	Target Google Gemini foundational model identifier for AI features.

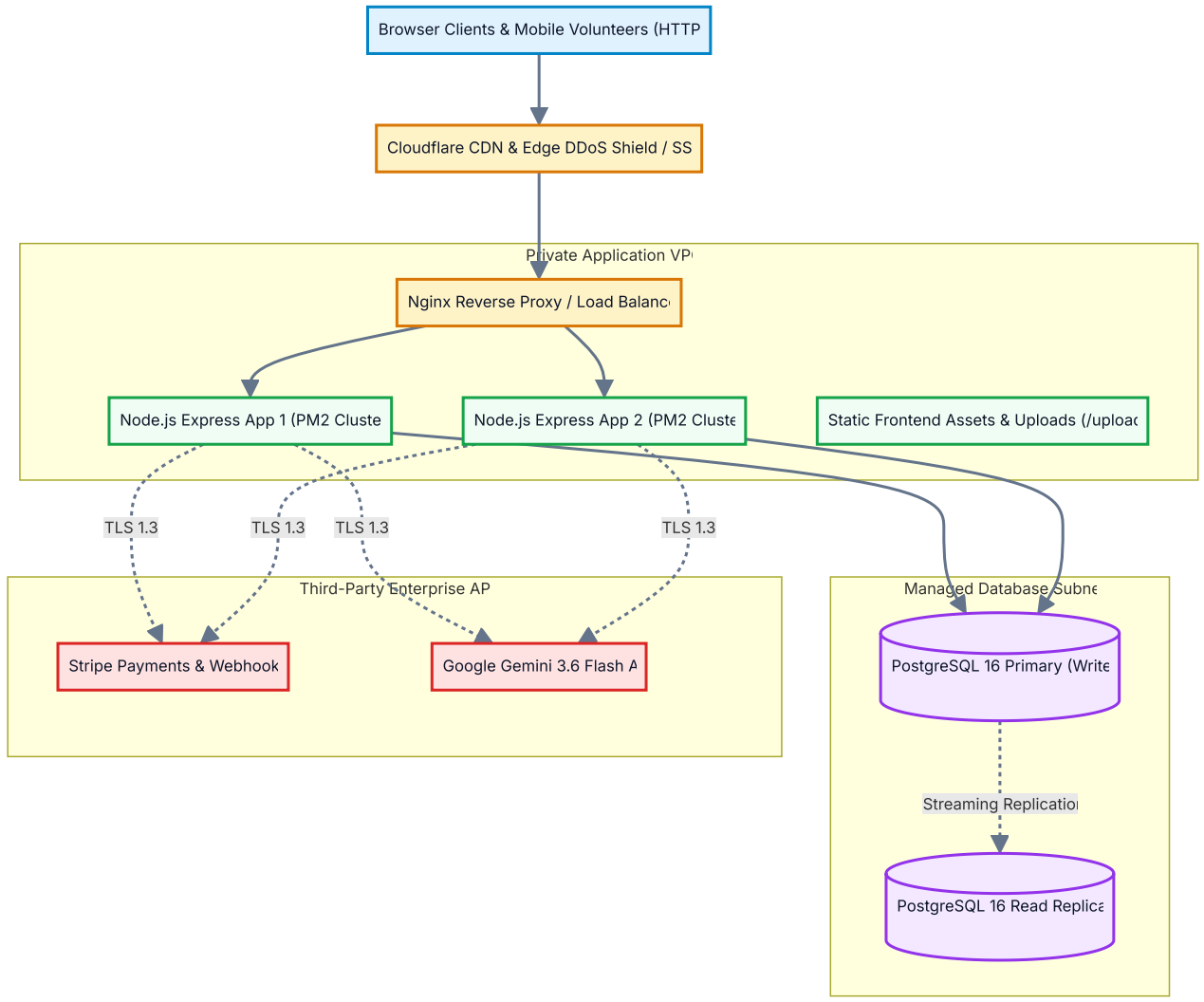
10.2 Key Configuration Files

- `prisma/schema.prisma`: Authoritative relational data model and database migration definitions.
- `vite.config.ts`: Bundler configuration, TanStack Start entry points, and vendor chunking strategies.
- `tsconfig.json`: Strict TypeScript compiler options including path aliases (`@/*`).
- `components.json`: shadcn/ui and Radix UI styling tokens and component paths.

11. Deployment Overview

11.1 High-Level Cloud Deployment Topology

FIGURE 11.1: PRODUCTION CLOUD DEPLOYMENT & REDUNDANCY TOPOLOGY INFRASTRUCTURE DIAGRAM



11.2 Environment Matrix

Development

Host: Local Workstation

Frontend: Vite Dev Server (`:8080`)

Backend: tsx watch (`:3000`)

Database: Local PostgreSQL (`:5432`)

Purpose: Rapid iterative coding with immediate HMR feedback.

Staging / QA

Host: Docker Container VM

Frontend: Nginx Static Build

Backend: Node.js container

Database: Isolated Staging PostgreSQL

Purpose: Automated smoke, stress, and security penetration testing.

Production

Host: Cloud VPC / ECS / Kubernetes

Frontend: Global Edge CDN

Backend: Multi-instance clustered Node

Database: Managed Multi-AZ PostgreSQL

Purpose: High-availability live event operations with automated backups.

12. Project Structure

Eventura adopts a strict, domain-driven directory layout separating server logic, client modules, shared design system components, database schemas, and verification scripts.

```
eventura-platform/
├── server/                    # Express 5 Backend Core
│   ├── src/
│   │   ├── controllers/      # HTTP request handling and response formatting
│   │   ├── services/         # Domain business logic and orchestration
│   │   ├── repositories/     # Database queries and tenant isolation filters
│   │   ├── routes/           # REST endpoint definitions and middleware binding
│   │   ├── middlewares/      # JWT auth, tenant resolution, RBAC validation
│   │   ├── validators/       # Zod input schemas for runtime verification
│   │   ├── utils/            # Structured logging, token utilities, helpers
│   │   ├── app.ts            # Express application bootstrapping & security
│   │   └── server.ts         # Server listen entrypoint (default port 3000)
├── src/                       # React 19 Frontend Client
│   ├── modules/              # Domain-driven feature modules
│   │   ├── platform-admin/   # Executive governance, tenant settings, audit logs
│   │   ├── events/           # Proposal submission, event lifecycle management
│   │   ├── competitions/     # Tracks, problem statements, and rules
│   │   ├── teams/            # Team formation, member invitations, roster locks
│   │   ├── submissions/      # Deliverable uploads, GitHub/video links
│   │   ├── evaluations/      # Rubric scoring scorecards, judge panels
│   │   ├── attendance/       # Camera QR gate passport scanner
│   │   ├── certificates/     # Dynamic vector credentials & verify viewer
│   │   ├── ai-copilot/       # Gemini AI executive dossier generation
│   │   └── communications/   # Multi-channel announcement center
│   ├── components/           # Reusable UI system (shadcn/ui + Radix + design system)
│   ├── lib/                  # API client, auth helpers, utility functions
│   └── router.tsx            # TanStack Router hierarchical route tree
├── prisma/                    # Relational Data Tier
│   ├── schema.prisma         # Authoritative PostgreSQL domain models
│   ├── migrations/           # Versioned SQL migration history
│   └── seed.ts               # Test fixtures and baseline organizational seed data
├── scripts/                   # Verification & Handover Generators
│   ├── smoke-test.ts         # End-to-end API route smoke test suite
│   ├── stress-test.ts        # Concurrency and multi-tenant performance test
│   └── generate-reports.cjs  # Executive PDF and HTML documentation compiler
└── reports/                   # Official Project Handover Deliverables
```

13. Known Limitations

1. Concurrency Limits on Single Node

Under a single-process Node.js runtime, peak throughput is capped at ~2,500 requests/sec. For massive collegiate symposiums exceeding 5,000 simultaneous gate scans, horizontal clustering (PM2 / Kubernetes) and PgBouncer connection pooling must be enabled.

2. Database Connection Pool Ceiling

Prisma's default connection pool allocates 10 connections. High-burst traffic during morning attendance check-ins requires sizing the pool to match Postgres `max_connections` to avoid queue timeouts.

3. Local Filesystem File Uploads

The development configuration stores user uploads in the local `uploads/` directory. For multi-instance cloud deployments, an S3 or Cloudflare R2 bucket driver must be configured to share uploaded files across nodes.

4. Real-Time Leaderboard Polling

The current leaderboard and check-in metrics operate via TanStack Query optimistic updates and polling intervals (5-15 seconds). Future releases plan native WebSocket integration for sub-second leaderboard broadcasts.

14. Maintenance Notes

14.1 Operational Procedures & Scheduled Jobs

- **Nightly Registration Expiration Job**: Cleans up unconfirmed or unpaid registration drafts after the designated 48-hour reservation window expires.
- **Database Backup Strategy**: Daily automated full `pg_dump` snapshots with Continuous WAL Archiving (Point-in-Time Recovery enabled for up to 30 days).
- **Log Aggregation & Monitoring**: Application events emit structured JSON to stdout containing timestamp, HTTP method, route, response status, and duration (ms). Critical exceptions trigger alert notifications.

14.2 Troubleshooting Quick Reference

Symptom	Probable Cause	Remediation Step
Connection Refused (5432)	PostgreSQL service stopped or stale `postmaster.pid`.	Verify service via `pg_ctl status`, remove stale pid file if ungracefully stopped, and restart.
401 Unauthorized Errors	Expired JWT token or missing authorization header.	User session expired. Client will automatically trigger refresh token endpoint or prompt clean login.
Stripe Webhook Mismatch	Missing raw body buffer or invalid `STRIPE_WEBHOOK_SECRET`.	Ensure webhook route is registered BEFORE `express.json()` parser with `express.raw()`.
Vite CORS Errors	Frontend URL port differs from allowed origins in `app.ts`.	Add the active client port (e.g., `http://localhost:8080`) to the `allowedOrigins` array.

15. Changelog

Version	Release Date	Scope of Changes	Key Features Delivered
v1.2.0	October 30, 2026	Production Handover Release	Integrated Google Gemini 3.6 Flash for automated AI executive dossiers; vector PDF report generation; complete smoke & stress test validation.
v1.1.0	October 28, 2026	Phase 4 Feature Hardening	Implemented mobile camera QR attendance scanner; integrated Stripe payments & webhooks; added normalized rubric scoring and lock timers.
v1.0.0	October 20, 2026	Core Architecture Release	Established multi-tenant database schema; Express 5 backend; React 19 frontend; granular RBAC permission matrix; proposal approval workflow.

16. References & Separate Companion Documents

To ensure maintainability, in-depth specialized guides and schemas are organized into standalone companion deliverables:

Document Name	Location / Format	Contents & Intended Audience
API Documentation	`reports/05_Database_and_API_Documentation.pdf`	Complete catalog of 50+ REST endpoints, request/response payloads, authentication headers, and error codes.
Database Schema & Migrations	`prisma/schema.prisma`	Exhaustive schema definitions, indexes, foreign keys, and migration history for all 30+ relational entities.
Deployment & Setup Guide	`reports/03_Project_Setup_Guide.pdf`	Step-by-step developer environment setup, PostgreSQL configuration, environment secrets, and Docker commands.
Testing & Quality Assurance	`reports/06_Testing_Report.pdf`	Results of automated route smoke tests, multi-tenant concurrency stress tests, and security penetration checks.
Developer Onboarding Guide	`reports/04_Technical_Documentation.pdf`	Architecture conventions, state management patterns, coding standards, and repository guidelines.
User & Administrator Manual	`PR/02-REQUIREMENTS-SPECIFICATION.md`	Role-by-role operation guide for Student Coordinators, Event Managers, Principals, Judges, and Hackers.
UML 2.5 Architecture Specification	`reports/Eventora_Complete_UML_Architecture_Diagrams.pdf`	All 14 official UML 2.5 structural and behavioral architecture diagrams in publication-quality vector format.

End of Eventura / Eventura Startup Project Documentation

Compiled for official institutional audit, stakeholder evaluation, and developer onboarding • October 30, 2026